M291 · SUPPORT · S10 JSON

s10 — Ranger le contenu dans un fichier JSON

Module ICT 291 · Développer des interfaces utilisateurs (UI)

Médiamaticien·ne CFC · Semaine 10 · 3 périodes de 45 minutes

Cette semaine, vous écrivez le fichier qui contiendra tout le contenu de votre application : `data.json`. La semaine prochaine, votre page l'affichera.

> **NOTE**
> Deux fichiers. — Celui-ci est la version **cours** (on le parcourt ensemble). La version approfondie — plus de contexte, les pièges, le lien avec votre projet — est le fichier `s10-json-avance.pdf`.

La scène

Vous changez de téléphone. Vous lancez le transfert, et vos 412 contacts arrivent sur le nouvel appareil en quelques secondes : nom, numéro, adresse e-mail, chacun dans la bonne case.

Cela marche parce que chaque contact est rangé de la même façon : une fiche, avec les mêmes cases, qui portent les mêmes noms. Imaginez que la moitié de vos contacts aient une case « tél » et l'autre moitié une case « numéro ». Le nouveau téléphone ne cherche que « numéro ». Il recopie 206 contacts sans numéro, sans aucun message d'erreur.

L'idée

Votre application aura le même problème. Les cartes de l'écran Liste affichent un nom, un quartier, « ouvert » ou « fermé ». Si ces informations sont écrites directement dans le HTML, chaque nouvelle carte est un bloc de HTML à recopier, et chaque changement doit être fait trente fois.

On sépare donc deux choses :

- le **contenu** (les noms, les quartiers, les horaires), rangé dans un fichier de données ;
- la **page** (le HTML et le CSS), qui affiche ce contenu, quel qu'il soit.

Ce fichier de données s'écrit dans un format appelé **JSON**. JSON est une manière d'écrire des fiches dans un simple fichier texte. Le nom vient de *JavaScript Object Notation*, mais vous n'avez pas besoin de JavaScript pour l'écrire. Un fichier JSON s'ouvre dans VS Code comme un fichier `.md`.

Quatre mots suffisent pour lire tout fichier JSON :

- une **fiche** : tout ce que l'on sait sur **une** chose (un lieu, un objet, un contact). Elle s'écrit entre accolades `{ }`. Le terme technique est *objet* ;

- une **étiquette** : le nom d'une case de la fiche, par exemple `"nom"`. Elle s'écrit toujours entre guillemets doubles `" "`. Le terme technique est *clé* ;
- une **valeur** : ce qui est rangé dans la case (voir les trois sortes juste après) ;
- une **pile** : la liste de toutes les fiches, entre crochets `[ ]`, séparées par des virgules. Le terme technique est *tableau*.

Une étiquette avec sa valeur, par exemple `"pages": 96`, s'appelle une **propriété**. C'est le terme technique que vous lirez dans la documentation et en semaine 11. Dans ce cours, on continue à dire « étiquette » pour le nom de la case.

Une valeur est de l'une de ces trois sortes dans ce cours :

- un **texte**, entre guillemets doubles : `"Café de la Gare"` ;
- un **nombre**, sans guillemets, avec un point pour les décimales : `2` ou `12.50` ;
- un **vrai/faux**, sans guillemets et en minuscules : `true` ou `false`. Le terme technique est *booléen*.

Un exemple : d'un livre à une fiche

La boîte à livres d'un quartier note ceci sur une fiche papier : « Le Petit Prince, d'Antoine de Saint-Exupéry, 1943, 96 pages, disponible ». Pour en faire une fiche JSON, on donne un nom à chaque information, puis on choisit la sorte de chaque valeur :

Sur le papier	Étiquette	Valeur	Sorte
Le Petit Prince	`titre`	`"Le Petit Prince"`	texte
d'Antoine de Saint-Exupéry	`auteur`	`"Antoine de Saint-Exupéry"`	texte
1943	`annee`	`1943`	nombre
96 pages	`pages`	`96`	nombre
disponible	`disponible`	`true`	vrai/faux

On ajoute un `id`, un numéro différent pour chaque livre, et on obtient une fiche :

```
{
"id": 1,
"titre": "Le Petit Prince",
"auteur": "Antoine de Saint-Exupéry",
"annee": 1943,
"pages": 96,
"disponible": true
}
```

Un deuxième livre aura **les mêmes étiquettes**, avec d'autres valeurs. Les deux fiches, entre `[ ]` et séparées par une virgule, forment une pile. Une recette, un film ou un instrument se rangent de la même façon : ce sont les exercices 2 et 3.

Le geste : lire une pile

Ouvrez `structure.exemple.json` dans VS Code. Il est dans le dossier de départ de l'exercice 4 : `02-exercices/s10/4-ma-base-json/depart/`. Il montre deux fiches de Phare, l'application exemple du cours (les endroits calmes encore ouverts ce soir, près de chez soi). Vous voyez ceci :

```
[
{
"id": 1,
"nom": "Café de la Gare",
"quartier": "Centre",
"ouvert": true,
"bruit": 2,
"resume": "Grandes tables, prises électriques, ferme à 22 h.",
"horaire": "Tous les jours, 6 h – 22 h"
},
{
"id": 2,
"nom": "Salle 204",
"quartier": "École",
"ouvert": false,
"bruit": 1,
"resume": "Salle de projet, ouverte les soirs de cours.",
"horaire": "Lundi et jeudi, 18 h – 21 h"
}
]
```

Lisez-le de l'extérieur vers l'intérieur :

1. Le `[` de la première ligne ouvre la pile. Le `]` de la dernière ligne la ferme.
2. Chaque `{ … }` est une fiche. Il y en a deux, séparées par une virgule après la première accolade fermante `},`.
3. Dans chaque fiche, une ligne = une étiquette, deux-points, une valeur. Une virgule sépare les lignes, sauf après la dernière.
4. Les deux fiches portent **exactement** les mêmes étiquettes, dans le même ordre. C'est ce qui permettra à votre page de les afficher toutes de la même façon.

Les trois règles qui cassent un fichier

Un fichier JSON est lu par un programme, et ce programme ne devine rien. Une seule faute, et il refuse **tout** le fichier : votre page n'affichera aucune carte, pas seulement la carte fautive.

1. **Guillemets doubles** autour de chaque étiquette et de chaque texte. L'apostrophe `'` ne compte pas comme guillemet.

2. **Virgules** : une virgule entre deux lignes et entre deux fiches. **Aucune** après la dernière ligne d'une fiche, ni après la dernière fiche.
3. **Ce qui s'ouvre se ferme** : chaque `{` a son `}`, chaque `[` a son `]`.

Ces règles ne sont pas une habitude du cours. Ce sont celles de la norme JSON (RFC 8259).

Comment savoir que votre fichier est juste

Deux signaux, à utiliser ensemble.

- **Dans VS Code**, une faute est soulignée d'une vague rouge, à l'endroit où le programme s'est perdu. La vraie faute est souvent juste **avant** le souligné : une virgule oubliée à la fin de la ligne précédente, par exemple.
- **Avec l'outil de contrôle** `controle-json.html`, fourni dans le dossier de départ des quatre exercices : double-cliquez dessus, il s'ouvre dans votre navigateur. Collez votre fichier, cliquez sur « Contrôler ». Il vous donne la ligne de la première faute, en français, puis il vérifie que toutes vos fiches portent les mêmes étiquettes, ce que VS Code ne fait pas. Il fonctionne sans Internet.

Le fichier est juste quand l'outil affiche, en vert : « Valide : N fiches, toutes avec les mêmes étiquettes. » Si votre fichier commence par `{`, l'outil vous dit en jaune que c'est une fiche seule, pas une pile.

> **NOTE**
> Pour vérifier ailleurs, avec des données inventées seulement. —
> - **JSONLint** (`https://jsonlint.com`) : un site où l'on colle du JSON. Il indique la ligne de la faute, en anglais. Il ne vérifie pas que vos fiches portent les mêmes étiquettes.
> - **La visionneuse JSON de Firefox** : ouvrez un fichier `.json` dans Firefox (glissez-le dans la fenêtre). S'il est juste, Firefox l'affiche en arbre, avec des fiches que l'on peut replier. S'il est cassé, il affiche en rouge un message comme « JSON.parse: expected double-quoted property name at line 17 column 3 ».
> - **Lire ce message** : MDN, le site de documentation du web de Mozilla, a une page en français, « SyntaxError: JSON.parse: bad parsing » (`https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Errors/JSON_bad_parse`). Elle liste ces messages et leur cause, par exemple une virgule en trop ou une apostrophe à la place d'un guillemet.

> **NOTE**
> Données personnelles. — Ne collez jamais de données personnelles (noms, téléphones, adresses, notes) dans un site en ligne, même un validateur. Ce que vous collez part sur un serveur que vous ne connaissez pas. Pour votre projet, utilisez `controle-json.html` : il fonctionne sans Internet et rien ne sort de votre ordinateur.

En semaine 11, vous verrez votre `data.json` arriver dans la page, dans l'onglet **Réseau** des outils de développement du navigateur.

Vérifiez-vous

1. Dans `"ouvert": true`, quelle est l'étiquette, quelle est la valeur, et de quelle sorte est la valeur ?
2. Une fiche se termine par `"bruit": 2,` puis `}`. Que dit l'outil de contrôle, et que corrigez-vous ?
3. Une camarade écrit `"ouvert": "true"`. Le fichier est valide. Pourquoi est-ce quand même un problème ?

Réponses : (1) étiquette `ouvert`, valeur `true`, un vrai/faux. (2) « Virgule en trop » : la dernière ligne de la fiche devient `"bruit": 2`, sans virgule. (3) Avec des guillemets, `"true"` est un **texte**, pas un vrai/faux. L'outil le signale en jaune. Et un texte non vide compte comme vrai pour la page : `"false"` aussi, si bien qu'un lieu fermé s'afficherait « ouvert ».

Les exercices de la semaine

Un **brief** est le document qui décrit une application : ses écrans et ce que chaque écran affiche. Le vôtre date de la semaine 5 ; l'exercice 3 s'entraîne sur un mini-brief court.

Exercice	Quand	Dans le pack
1 — La pile cassée : réparer un fichier JSON écrit par quelqu'un d'autre	P1, 30 min, seul	`02-exercices/s10/1-la-pile-cassee.pdf` et le dossier `02-exercices/s10/1-la-pile-cassee/depart/`
2 — Une chose, une fiche : transformer un disque, une recette et des films en fiches, puis en piles	P2, 20 min, seul	`02-exercices/s10/2-une-chose-une-fiche.pdf` et le dossier `02-exercices/s10/2-une-chose-une-fiche/depart/`
3 — Trouver les étiquettes : tirer les étiquettes d'un mini-brief avec une méthode en 6 questions	P2, 20 min seul, puis 3 min en binôme	`02-exercices/s10/3-trouver-les-etiquettes.pdf` et le dossier `02-exercices/s10/3-trouver-les-etiquettes/depart/`
4 — Ma base JSON : écrire votre `data.json`, 5 fiches à la main, puis 30 avec l'IA	P3 avec votre brief (étapes 1 à 5), puis à la maison avant la semaine 11 (étapes 6 à 8)	`02-exercices/s10/4-ma-base-json.pdf` et le dossier `02-exercices/s10/4-ma-base-json/depart/`

Les quatre exercices ont deux parcours. Le parcours **essentiel** suit les étapes. Le parcours **avancé** fait les mêmes étapes, puis la partie « Pour les rapides » de chaque énoncé.

Avec l'IA

Cette semaine, l'IA n'écrit rien avant que vous ayez écrit vous-même. Exercices 1 et 2 : pas d'IA, vous apprenez à lire une faute et à choisir la sorte d'une valeur. Exercice 3, à la maison après le cours : l'IA relit votre liste d'étiquettes, et c'est vous qui gardez ou refusez ses suggestions. Exercice 4 : elle prolonge vos 5 fiches jusqu'à 30, puis vous contrôlez avec `controle-json.html` et vous relisez. Une IA ajoute volontiers des étiquettes « au cas où » et écrit parfois `"true"` entre guillemets. Ne lui donnez jamais de données de vraies personnes.

Pour les rapides

Chaque énoncé se termine par sa partie « Pour les rapides » : un disque à deux artistes (exercice 2), une ludothèque (exercice 3), un horaire en sous-fiche ou 40 fiches (exercice 4). Elle n'est pas notée en plus. La version approfondie, `s10-json-avance.pdf`, explique les sous-fiches et le schéma.

Lien avec la pratique

- **Semaine 11** : votre page lit `data.json` et affiche une carte par fiche. Elle n'affiche que les étiquettes que vous aurez choisies aujourd'hui.
- **Semaine 12** : le filtre et la recherche ne peuvent porter que sur une étiquette qui existe, écrite de la même façon dans toutes les fiches.
- **Note 3** : le cahier des charges du projet final demande un `data.json` **personnel** d'au moins 30 fiches. Le vôtre, tiré de votre brief, pas celui du voisin.
- **Dans le métier**, un client vous enverra un jour un tableau Excel « à mettre sur le site ». Vous ferez le même travail : choisir les colonnes, vérifier que chaque ligne les remplit de la même façon.